

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 2. Memoria y procesos

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 12 de septiembre del 2026

Índice

1. Fundamentos y taxonomía de la memoria en los sistemas de cómputo 1
1.1. Definición formal y rol en la arquitectura de von Neumann 1
1.2. Jerarquía de memoria y principios de localidad 1
1.3. Criterios de clasificación y tipos de memoria 3

2. Evolución histórica de la memoria principal y tecnologías RAM 5
2.1. Hitos pioneros del almacenamiento primario 5
2.2. Celdas de memoria de acceso aleatorio: SRAM vs. DRAM 7
2.3. Evolución de las tecnologías DRAM sincrónicas y de alta tasa de transferencia 8

3. Evolución histórica de la memoria secundaria y almacenamiento masivo 11
3.1. Tecnologías electromecánicas y magnéticas tempranas 11
3.2. Medios ópticos y consolidación del disco duro magnético (HDD) 11
3.3. Tecnologías de estado sólido y almacenamiento no volátil contemporáneo . . 13

4. Memoria de intercambio (Swapping) y gestión de memoria virtual 16
4.1. Definición y distinción técnica: intercambio de procesos frente a paginación bajo demanda 16
4.2. Funciones y operatividad de la memoria de intercambio en el sistema operativo 16
4.3. Rendimiento, degradación por hiperpaginación (*Thrashing*) y mecanismos de mitigación 17

5. Arquitectura y ciclo de vida de los procesos 18
5.1. Definición formal de proceso y organización en memoria 18
5.2. Estructura y características del proceso: Bloque de Control de Proceso (PCB) 18
5.3. Modelo de estados y transiciones en la ejecución del proceso 19

6. Hilos de ejecución y análisis comparativo frente a procesos 20
6.1. Definición formal de hilo (*Thread*) y estructura de control (TCB) 20
6.2. Modelos de gestión de hilos y correspondencia de planificación 20
6.3. Estudio comparativo y diferencial entre procesos e hilos 21

7. Conclusiones personales 23

Referencias 23

1. Fundamentos y taxonomía de la memoria en los sistemas de cómputo

1.1. Definición formal y rol en la arquitectura de von Neumann

En la teoría de sistemas computacionales y la ingeniería de computadores, la memoria no constituye un mero repositorio pasivo de información, sino un subsistema funcional activo y estructurado concebido para almacenar estados binarios transitorios o permanentes organizados en celdas direccionables numéricamente (Stallings, 2018, §1.1, p. 31). Desde la formulación seminal del modelo de John von Neumann en 1945, la memoria principal ocupa el núcleo operativo del procesamiento automático de datos mediante el principio de programa almacenado (*stored-program concept*). Bajo este paradigma, las instrucciones de control del programa y los datos sobre los cuales operan dichas instrucciones residen indistintamente en el mismo medio físico de almacenamiento y comparten un espacio de direccionamiento lineal homogéneo (Silberschatz et al., 2006, §1.2, p. 8).

La interacción entre la unidad central de procesamiento (CPU) y la memoria opera mediante registros dedicados y buses del sistema. La CPU gobierna los accesos con el registro de direcciones de memoria (*memory address register*, MAR), que fija la coordenada numérica de la celda, y el registro de búfer o datos (*memory buffer register*, MBR), que sostiene temporalmente la palabra transferida (Stallings, 2018, §1.1, p. 31). Esta transferencia se realiza a través del bus de direcciones (cuyo ancho fija el espacio direccionable), el bus de datos (ancho de palabra transferible) y el bus de control (señales de temporización y selección de lectura/escritura) (Deitel et al., 2004, §2.3, p. 64). El procesamiento se rige por el ciclo de instrucción-ejecución (*fetch-decode-execute-store*): el contador de programa (PC) carga la dirección en el MAR para transferir la instrucción al MBR y al registro de instrucción (IR) (*fetch*); la unidad de control decodifica el código de operación (*decode*); la unidad aritmético-lógica (ALU) opera sobre los operandos (*execute*); y los resultados se escriben en memoria o registros (*store*) (Silberschatz et al., 2006, §1.2, p. 8).

La memoria es agnóstica a la semántica de los datos binarios que aloja. Al unificarse el bus de instrucciones y datos en von Neumann, surge el “cuello de botella de von Neumann” (*von Neumann bottleneck*) (Deitel et al., 2004, §2.3, p. 64): como la velocidad de la CPU escaló históricamente más rápido que la latencia de la DRAM, el procesador inserta ciclos de espera (*wait states*) aguardando datos. En contraste, la arquitectura Harvard separa físicamente los buses de instrucciones y datos; aunque más costosa para memoria principal, esta separación se preserva internamente en las cachés de nivel 1 (L1i y L1d), manteniendo una interfaz unificada hacia el exterior (Stallings, 2018, §1.6, p. 49).

1.2. Jerarquía de memoria y principios de localidad

El diseño del subsistema de memoria balancea tres restricciones físicas y económicas: capacidad, tiempo de acceso y costo por bit (Stallings, 2018, §1.5, pp. 45–46). A menor tiempo de acceso, mayor es el costo monetario por bit; a mayor capacidad, menor es el costo unitario, pero mayor es la latencia por retardos de decodificación y líneas físicas (Deitel et al., 2004, §2.3, p. 64). Para compatibilizar velocidad y costo masivo, los sistemas organizan el almacenamiento en una pirámide jerárquica escalonada (Stallings, 2018, §1.5, p. 47). En la cúspide se ubican los registros internos de la CPU (fracciones de ciclo, cientos de bytes). Le sigue la memoria caché multinivel (L1, L2, L3) en tecnología estática (*static*

random-access memory, SRAM), con latencias de 2 a 40 ciclos y capacidades de kilobytes a megabytes (Deitel et al., 2004, §2.3, p. 64). Debajo se posiciona la memoria principal en tecnología dinámica (*dynamic random-access memory*, DRAM), que aloja los procesos activos con latencias de decenas de nanosegundos y capacidades de gigabytes. En la base se sitúa el almacenamiento secundario no volátil (HDD, SSD Flash NAND) y terciario (cintas, ópticos), brindando terabytes a bajo costo con latencias de microsegundos a milisegundos (Silberschatz et al., 2006, §1.8, pp. 21–23). Esta pirámide se ilustra en la Img. 1.

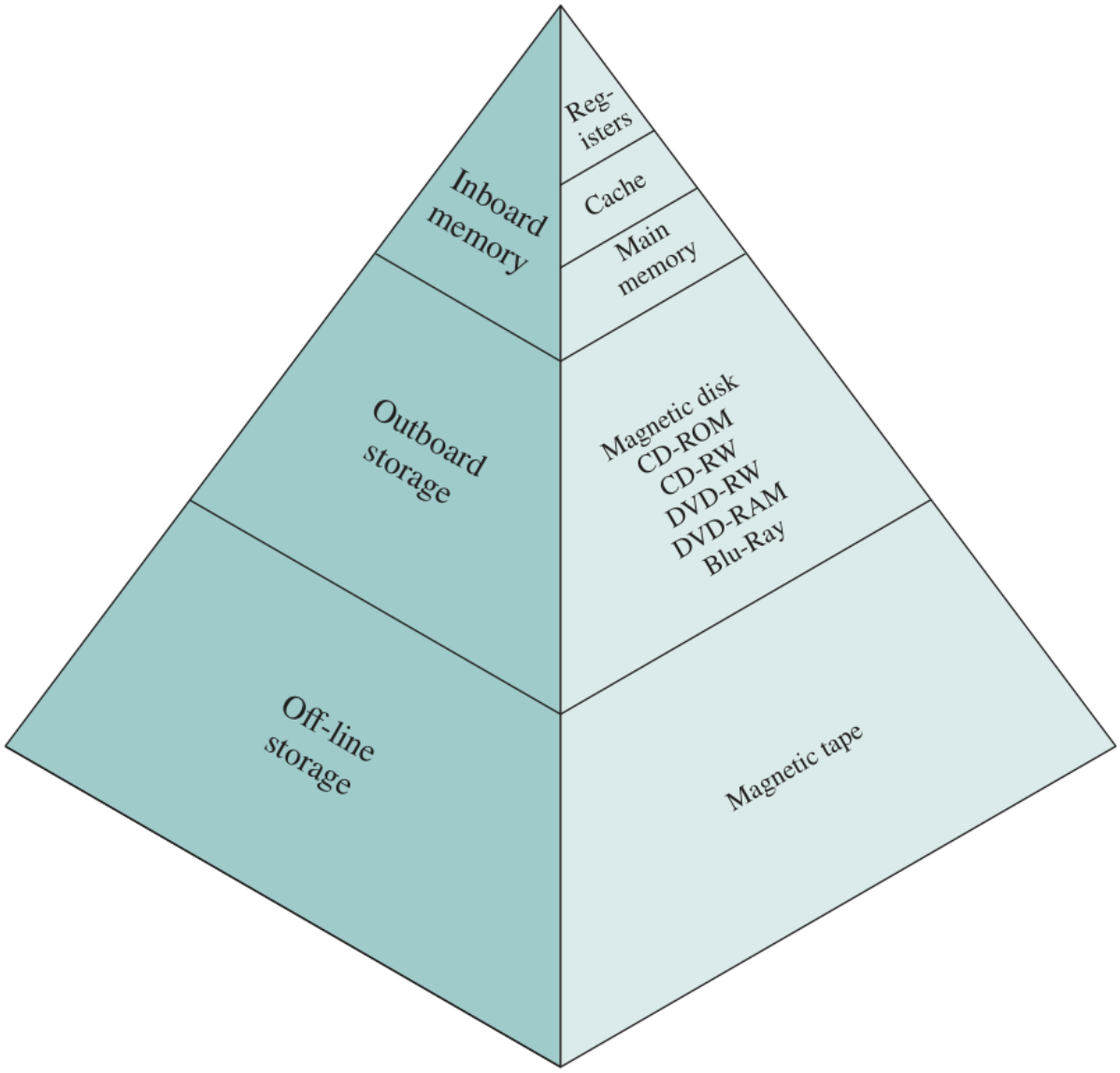


Imagen 1: Niveles jerárquicos de memoria, compensación costo-rendimiento y tiempos de acceso (Stallings, 2018, §1.5, p. 47).

La operatividad y viabilidad técnica de esta jerarquía descansa en el principio de localidad de referencia (*locality of reference*), formulado formalmente por Peter J. Denning en 1968 (Stallings, 2018, §1.5, p. 48). Este postulado empírico establece que, durante la ejecución de un programa informático, las referencias a memoria efectuadas por la CPU tienden a agruparse en regiones o bloques sumamente acotados del espacio de direcciones durante intervalos de tiempo determinados. El fenómeno se descompone en dos propiedades fundamentales:

- **Localidad temporal:** Si una posición de memoria concreta es accedida en un instante t, existe una probabilidad cuantitativamente elevada de que sea referenciada nuevamente en un instante cercano $t + \Delta t$. Este comportamiento es típico de estructuras de bucles iterativos (*loops*), variables acumuladoras, contadores y subrutinas invocadas con frecuencia en el cuerpo de un algoritmo (Stallings, 2018, §1.5, p. 48).
- **Localidad espacial:** Si una dirección de memoria es referenciada en un instante dado, existe una alta probabilidad de que las posiciones continuas y adyacentes a ella sean accedidas en un futuro inmediato. Dicha propiedad es característica del procesamiento secuencial de instrucciones en el código de máquina, el recorrido de arreglos, vectores unidimensionales y matrices de datos, así como el crecimiento contiguo de marcos en la pila de llamadas (*call stack*) (Stallings, 2018, §1.5, p. 48).

El aprovechamiento coordinado de la localidad de referencia permite formular matemáticamente el tiempo promedio de acceso efectivo a memoria (T_{ef}) en una estructura de múltiples niveles de almacenamiento. Para un esquema simplificado de dos niveles representativo de la relación entre una memoria de nivel 1 (como una memoria caché) y una memoria de nivel 2 (como la memoria principal DRAM), el tiempo efectivo queda definido por:

$$T_{\text{ef}} = H \cdot T_1 + (1 - H) \cdot (T_1 + T_2) \tag{1}$$

donde H representa la tasa de aciertos (*hit ratio*), correspondiente a la fracción de accesos resueltos satisfactoriamente en el nivel más rápido; T_1 es la latencia de acceso al nivel 1; y T_2 es la penalización temporal de acceso al nivel 2 cuando ocurre un fallo (*miss*) (Stallings, 2018, §1.5, p. 48).

Cuando la tasa de aciertos H alcanza valores típicos de diseño superiores al 95 % ($H \geqslant 0.95$), el segundo término de la Ecuación (1) se reduce drásticamente, haciendo que el tiempo de acceso promedio percibido por el procesador converja estrechamente hacia la velocidad de la memoria caché rápida T_1, mientras que el sistema en su conjunto exhibe la capacidad masiva y el costo económico favorable del nivel inferior T_2.

1.3. Criterios de clasificación y tipos de memoria

El espectro de tecnologías de memoria se clasifica formalmente según cuatro criterios ortogonales: método de acceso físico, persistencia de estados, soporte material de fabricación y mutabilidad operativa (Silberschatz et al., 2006, §1.2, pp. 8–10; Deitel et al., 2004, §2.3, pp. 64–65). Por la disciplina de acceso, la memoria de acceso aleatorio (*random access*) ofrece latencia constante por decodificación matricial directa (SRAM, DRAM, ROM) (Deitel et al., 2004, §2.3, p. 65); el acceso secuencial exige un recorrido lineal continuo con latencia dependiente de la posición previa (cintas magnéticas) (Silberschatz et al., 2006, §1.8, p. 21); el acceso directo combina saltos radiales a pistas con rotación sectorial (HDD) (Silberschatz et al., 2006, §1.8, p. 22); y el acceso asociativo (*content-addressable*) compara patrones en paralelo en un ciclo (TLB y etiquetas de caché) (Stallings, 2018, §8.1, p. 384).

Por estabilidad energética, los medios volátiles disipan su contenido ante cortes de tensión (DRAM, SRAM), mientras que los no volátiles retienen polarización magnética, trampas de carga en compuertas flotantes o estados cristalinos sin energía externa (ROM, Flash, HDD). Tecnologías híbridas como la NVRAM (*non-volatile RAM*) respaldan matrices DRAM con baterías o supercondensadores que vuelcan datos a Flash ante fallas de alimentación (Silberschatz et al., 2006, §1.2, p. 9). Por soporte y mutabilidad, coexisten microelectrónica

CMOS (biestables 6T y celdas 1T1C), sustratos magnéticos (platos de HDD, cintas), medios ópticos por difracción láser (CD, DVD, BD) y memorias de cambio de fase, abarcando desde dispositivos de lectura/escritura simétrica (RWM) hasta memorias no reescribibles de máscara (Mask ROM), programables (PROM) y borrables eléctricamente por bloques (EEPROM y Flash NAND/NOR) (Silberschatz et al., 2006, §1.2, pp. 8–10). La Tabla 1 sintetiza esta taxonomía multidimensional.

Tabla 1: Taxonomía analítica de las tecnologías de memoria en sistemas de cómputo.

Criterio de clasificación	Categoría o variante	Mecanismo físico y lógico	Aplicación característica en la arquitectura
Disciplina de acceso	Aleatorio (RAM) Secuencial Directo Asociativo	Decodificación matricial directa Recorrido físico lineal de cinta Salto radial a pista y rotación de sector Comparación de etiquetas en paralelo	Memoria principal DRAM y caché SRAM Resguardo masivo en cinta magnética Discos duros mecánicos HDD Búferes TLB de paginación y etiquetas de caché
Persistencia	Volátil No volátil NVRAM híbrida	Retención activa por cerrojo o condensador Atrapamiento de carga o dominios magnéticos Batería de litio/supercondensador y flash	Espacio de ejecución de procesos y caché L1–L3 Firmware BIOS/UEFI, almacenamiento secundario Servidores empresariales y búferes RAID seguros
Soporte físico	Semiconductora Magnética Óptica Cambio de fase	Transistores CMOS y condensadores integrados Polarización dipolar de granos ferromagnéticos Dispersión de haz láser sobre pozos microscópicos Transición cristalina amorfa reversible	Circuitos integrados de memoria RAM y caché Platos de discos duros y cintas LTO Discos compactos CD, DVD y Blu-ray Unidades 3D XPoint y memorias de clase almacenamiento
Mutabilidad	RWM ROM/PROM EEPROM Flash (NOR/NAND)	Ciclos simétricos de lectura y escritura rápida Configuración física de máscara o microfusibles Tunelamiento de electrones celda por celda Borrado masivo por bloques de transistores	Memoria de trabajo de la CPU y memoria principal Tablas de arranque fijas y microcódigo cableado Parámetros de configuración persistente en placas Almacenamiento masivo en SSD y almacenamiento embebido

2. Evolución histórica de la memoria principal y tecnologías RAM

2.1. Hitos pioneros del almacenamiento primario

El desarrollo histórico de la memoria principal en los sistemas computacionales ha estado marcado por la constante búsqueda de reducir los tiempos de latencia y maximizar la densidad espacial y la fiabilidad de retención binaria. Antes de la consolidación de la microelectrónica de silicio, los diseñadores de arquitecturas de primera generación recurrieron a fenómenos electrostáticos, acústicos y ferromagnéticos para materializar el almacenamiento primario direccionable (McHoes & Flynn, 2011, §2.1, p. 32).

El primer dispositivo capaz de proporcionar un almacenamiento binario electrónico con acceso aleatorio genuino fue el tubo Williams-Kilburn, patentado en 1947 por Freddie Williams y Tom Kilburn en la Universidad de Mánchester para la computadora Manchester Baby (Silberschatz et al., 2006, §1.2, p. 8). Este sistema empleaba un tubo de rayos catódicos (*cathode ray tube*, CRT) convencional en cuya pantalla fosforescente se disparaba un haz de electrones enfocado. El impacto inducía un fenómeno de emisión secundaria de electrones, creando un pequeño pozo de potencial con carga positiva o neutra en la superficie interna del tubo para representar los ceros y unos lógicos. Como la carga electrostática tendía a recombinarse y disiparse en fracciones de segundo, el haz debía explorar continuamente los puntos para regenerar el estado de cada bit, ofreciendo capacidades del orden de 1024 a 2048 bits con tiempos de acceso de decenas de microsegundos. La estructura de este dispositivo se ilustra en la Img. 2.

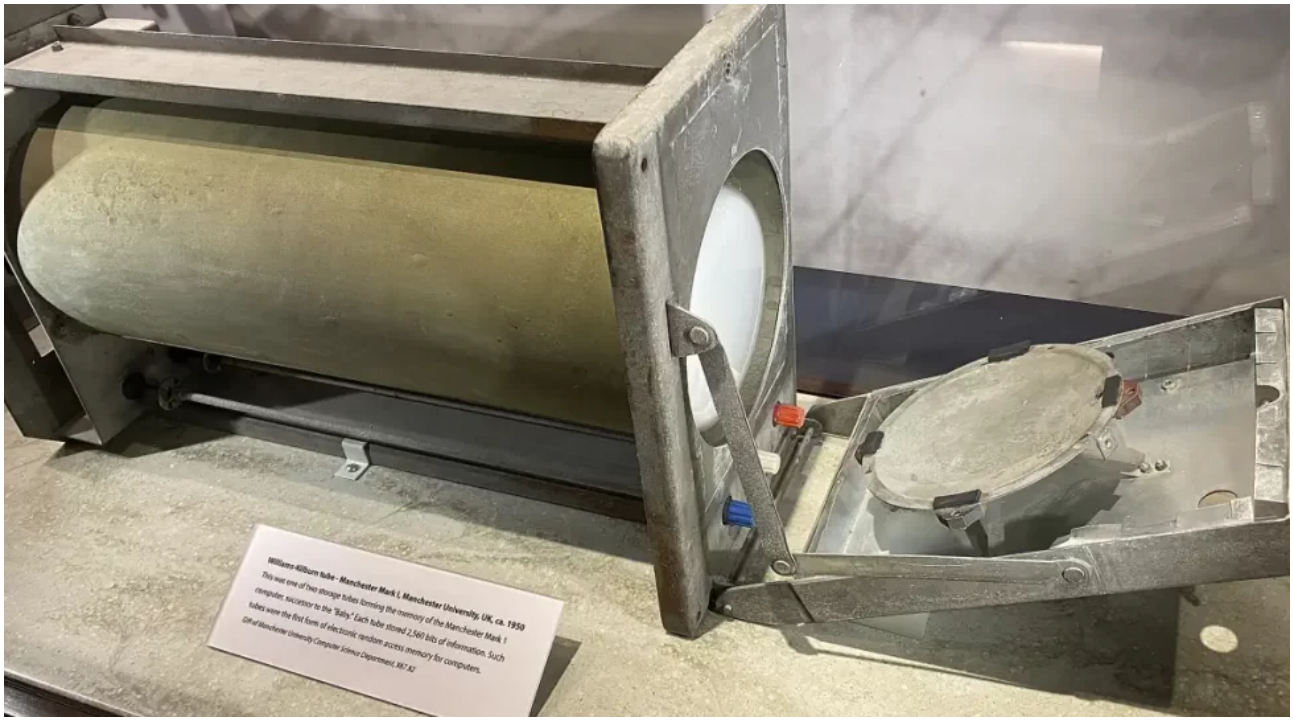


Imagen 2: Tubo Williams-Kilburn de almacenamiento electrostático de acceso aleatorio (1947) (Silberschatz et al., 2006, §1.2, p. 8).

De forma paralela, entre 1947 y 1949, J. Presper Eckert desarrolló para las computadoras EDVAC y UNIVAC I las líneas de retardo acústico de mercurio (*mercury delay lines*).

Este dispositivo electromecánico convertía trenes de impulsos eléctricos en ondas sonoras mecánicas mediante transductores piezoeléctricos de cuarzo situados en los extremos de tubos longitudinales rellenos de mercurio líquido. Las ondas tardaban un intervalo calibrado en viajar a lo largo del medio físico antes de ser reconvertidas en señales eléctricas en el extremo receptor, donde un circuito regenerador las amplificaba y realimentaba al emisor. Si bien este esquema permitía almacenar cientos de bits por columna de mercurio a bajo costo relativo, constituía un almacenamiento secuencial recirculante con tiempos de latencia sumamente elevados y una extrema sensibilidad a las variaciones térmicas que modificaban la velocidad acústica del medio.

La consagración de una memoria principal verdaderamente robusta, compacta y no volátil se alcanzó entre 1949 y 1953 con la invención de la memoria de núcleos magnéticos (*magnetic core memory*), ideada conceptualmente por An Wang y perfectionada arquitectónicamente por Jay Forrester en el proyecto Whirlwind del Instituto Tecnológico de Massachusetts (MIT) e integrada a gran escala por IBM (McHoes & Flynn, 2011, §2.1, p. 32). Como se aprecia en la Img. 3, esta memoria se componía de una matriz reticular de pequeños anillos toroidales de ferrita ferromagnética enhebrados en los cruces de conductores de selección coincidentes en los ejes horizontal (X) y vertical (Y).

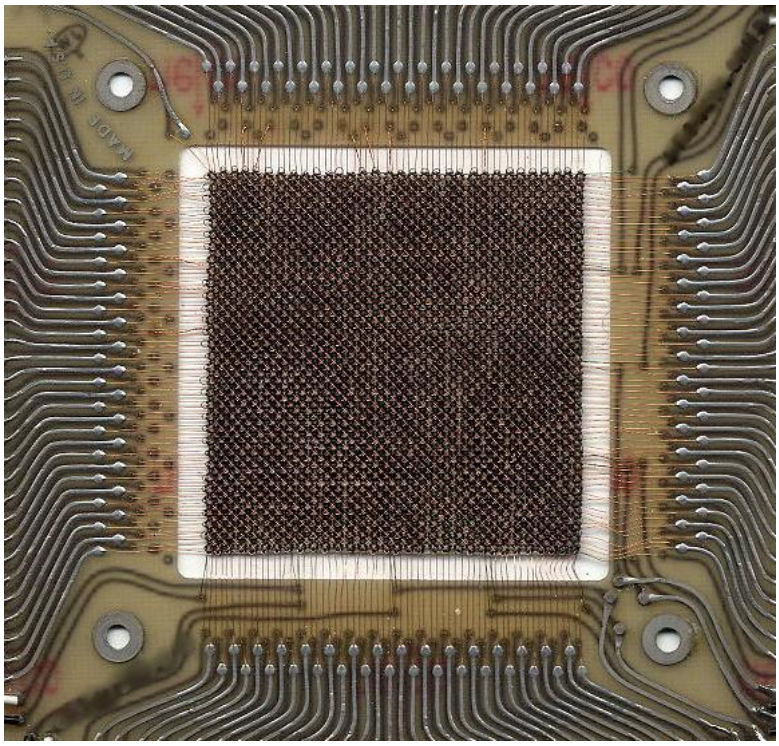


Imagen 3: Matriz de memoria de núcleos magnéticos toroidales con hilos de selección coincidentes (1953) (McHoes & Flynn, 2011, §2.1, p. 32).

El principio operativo de los núcleos de ferrita residía en su curva de histéresis magnética rectangular: al aplicar media corriente de conmutación ($\frac{1}{2}I_m$) simultáneamente en una línea X y una línea Y, solo el toroide ubicado en la intersección recibía la corriente magnetizante crítica I_m necesaria para invertir su orientación magnética dipolar remanente (en sentido horario o antihorario, codificando el cero o el uno lógico). Al conmutar el estado del núcleo durante la lectura, la variación de flujo magnético inducía una corriente en un hilo sensor común (*sense wire*). Dado que este proceso destruía la orientación previa del bit (lectura destructiva), el controlador requería forzosamente un ciclo complementario de

reescritura inmediata asistido por un hilo de inhibición (*inhibit wire*). La memoria de núcleos dominó la industria por más de dos décadas, otorgando al subsistema de memoria principal la denominación histórica de “núcleo” (core memory) que aún persiste en la terminología técnica (McHoes & Flynn, 2011, §2.1, p. 32).

El punto de inflexión definitivo hacia el almacenamiento moderno sobre silicio se produjo en 1968, cuando Robert H. Dennard concibió en los laboratorios de IBM la celda de memoria dinámica de un único transistor y un condensador (1T1C). Sobre esta base, en octubre de 1970 la corporación Intel lanzó al mercado el chip Intel 1103, presentado en la Img. 4. Fabricado con litografía semiconductora de óxido metálico de canal P (PMOS) y encapsulado en una pastilla DIP de 18 pines, el 1103 ofrecía una capacidad de 1024 bits (1 Kibit) a un costo por bit inferior al de los toroides de ferrita, desplazando para siempre a la memoria magnética como estándar de almacenamiento primario e iniciando la era de la memoria dinámica de semiconductores.

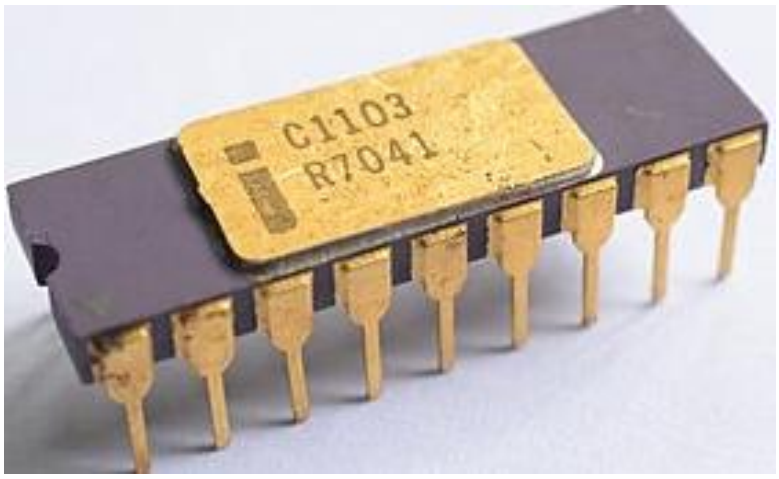


Imagen 4: Circuito integrado Intel 1103 en encapsulado DIP de 18 pines, primer chip comercial masivo de memoria DRAM de silicio (1970) (Deitel et al., 2004, §2.3, p. 65).

2.2. Celdas de memoria de acceso aleatorio: SRAM vs. DRAM

En la microelectrónica contemporánea coexisten dos arquitecturas fundamentales para la implementación de memorias de lectura y escritura de acceso aleatorio en circuitos integrados: la celda estática (SRAM) y la celda dinámica (DRAM) (Deitel et al., 2004, §2.3, p. 65). La divergencia entre ambas radica en la física del almacenamiento y en la topología circuital a nivel de transistor:

La celda básica de una memoria **SRAM** estándar se estructura a partir de seis transistores MOSFET (diseño 6T) formando dos inversores CMOS acoplados en cerrojo biestable (*bistable latch*) gobernado por la línea de palabra (*wordline*) y líneas de bit complementarias (*bitline* y $\overline{\textit{bitline}}$). Sus propiedades distintivas comprenden: retención continua sin refresco mientras exista alimentación; conmutación ultrarrápida en régimen subnanosegundo (latencias de 0.5 a 3 ns); y baja densidad con elevado costo por bit, destinándose exclusivamente a las memorias caché multinivel L1–L3 (Deitel et al., 2004, §2.3, p. 65).

Por su parte, la celda fundamental de una memoria **DRAM** sigue el diseño minimalista 1T1C de Dennard (un transistor de paso y un condensador de almacenamiento de carga $Q = C \cdot V$). Sus rasgos operativos críticos son: 1) necesidad imperativa de ciclos periódicos

de refresco (*refresh cycles*) cada 32 a 64 ms mediante pulsos automáticos (*auto-refresh*) gobernados por $\overline{\text{RAS}}$ y $\overline{\text{CAS}}$ para compensar fugas subumbral; 2) lectura destructiva restituida por amplificadores de sentido (*sense amplifiers*); y 3) máxima densidad de integración y mínimo costo unitario por gigabyte, erigiéndose en el estándar de memoria principal (Deitel et al., 2004, §2.3, p. 65).

2.3. Evolución de las tecnologías DRAM sincrónicas y de alta tasa de transferencia

Desde la década de 1980, la memoria dinámica evolucionó desde interfaces asíncronas lentas (FPM con retención de línea de fila y EDO con cerrojos de salida para solapar accesos) hacia la estandarización sincrónica por JEDEC con **SDR SDRAM** (*Single Data Rate Synchronous DRAM*, 1996), alineando comandos y transferencias al reloj del bus (66 a 133 MHz sobre módulos DIMM de 168 pines) (Deitel et al., 2004, §2.3, p. 65).

A partir del 2000, la tecnología de doble tasa de transferencia (*Double Data Rate*, DDR) impulsó el ancho de banda efectivo transfiriendo en ambos flancos de reloj y escalando progresivamente los búferes de prebúsqueda: DDR (2000, 2n-prefetch, 2.5 V, DIMM 184 pines); DDR2 (2003, 4n-prefetch, 1.8 V, terminación en chip ODT, DIMM 240 pines); DDR3 (2007, 8n-prefetch, 1.5 V / 1.35 V); DDR4 (2014, grupos de bancos independientes *Bank Groups*, 1.2 V, DIMM 288 pines); y DDR5 (2020), que dividió el módulo en dos subcanales de 32 bits, integró administración de potencia PMIC a 1.1 V, corrección de errores en troquel (*on-die ECC*) y prebúsqueda equivalente 16n (BL16). Especializaciones arquitectónicas abarcan la familia **LPDDR** (*Low Power DDR*) para dispositivos móviles y memorias apiladas 3D **HBM** (*High Bandwidth Memory*) con vías pasantes en silicio (TSV) para GPU y aceleradores neuronales.

La Img. 5 ilustra la evolución morfológica de los módulos DIMM estandarizados por JEDEC, y la Tabla 2 sintetiza la cronología integral de la memoria RAM.

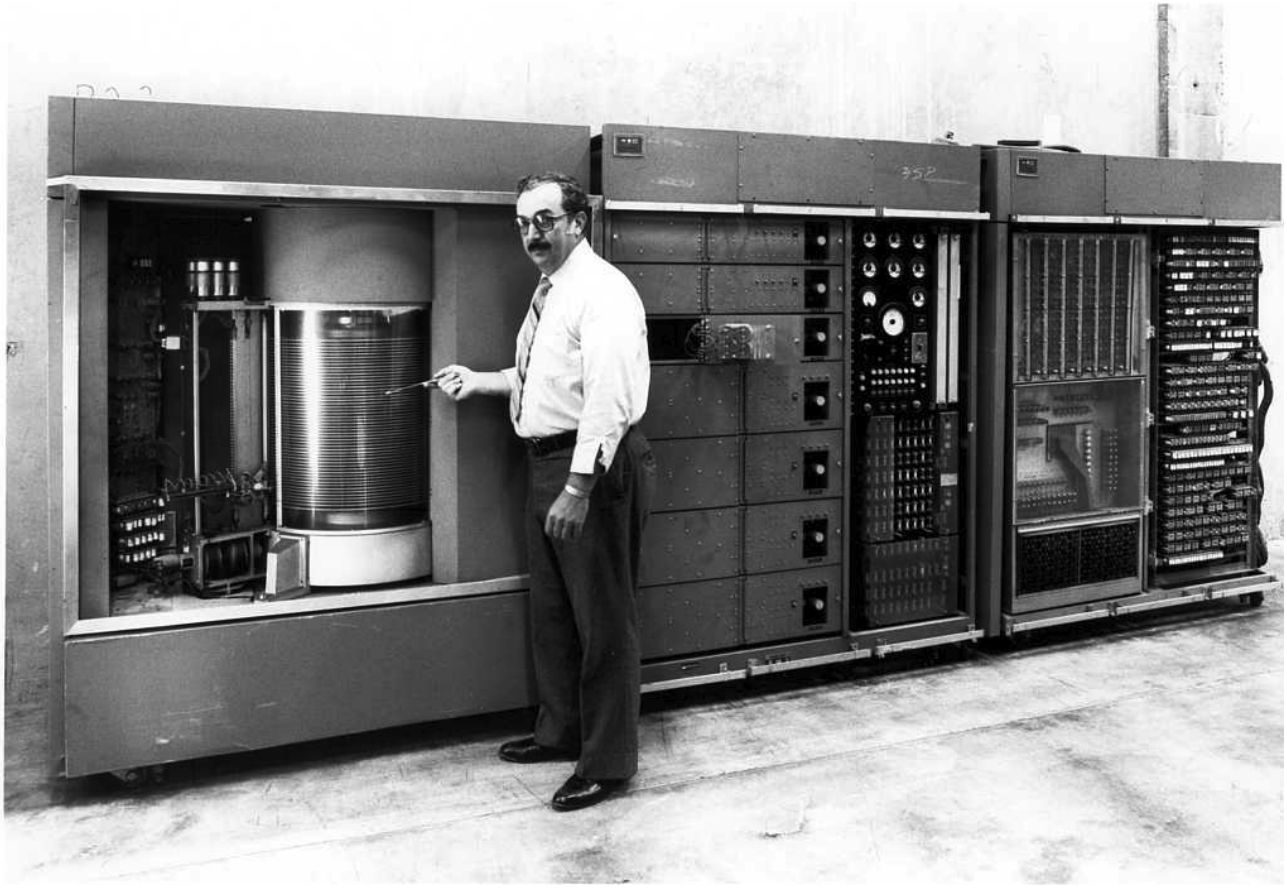


Imagen 5: Comparativa de factores de forma, muescas mecánicas y densidad de pines en módulos DIMM de SDRAM a DDR5 (Deitel et al., 2004, §2.3, p. 65).

Tabla 2: Cronología evolutiva integral de las tecnologías de memoria principal y RAM.

Año	Denominación	Topología física y celda	Tensión / Pines	Tasa o ancho de banda	Referencia de imagen / Hito
1947	Tubo Williams-Kilburn	Carga electrostática en CRT con emisión secundaria	Alta tensión / Nube analógica	≈ 1000 ops/seg (1–2 Kb)	Img. 2 Manchester Baby
1949	Línea de retardo	Trenes de ondas acústicas en tubos de mercurio	Transductor piezoeléctrico	≈ 1 MHz reloj Latencia $> 200\,\mu$s	EDVAC y UNIVAC I
1953	Núcleos magnéticos	Toroides de ferrita ferromagnética con selección X/Y	Corriente I_m / Hilos de cobre	Tiempos de ciclo de 1 a 6 μs	Img. 3 MIT Whirlwind / IBM
1970	Intel 1103	Circuito integrado DRAM planar de silicio en PMOS	16 V / DIP 18 pines	Tiempo de acceso de 300 ns (1 Kb)	Img. 4 Primera DRAM comercial
1985	FPM DRAM	DRAM asíncrona con retención de fila activa ($\overline{\text{RAS}}$)	5.0 V / SIMM 30–72 pines	Hasta 25 MHz Acceso a 60–70 ns	Sistemas 386 / 486
1994	EDO DRAM	Cerrojo de salida para solapamiento de ráfagas $\overline{\text{CAS}}$	5.0 V / 3.3 V SIMM 72 pines	Hasta 40 MHz Acceso a 50 ns	Pentium clásico
1996	SDR SDRAM	DRAM sincrónica acoplada al reloj del sistema (JEDEC)	3.3 V / DIMM 168 pines	66 a 133 MHz 0.53 a 1.06 GB/s	Img. 5 PC66, PC100, PC133
2000	DDR SDRAM	Doble tasa de datos (subida y bajada), prefetch de 2n	2.5 V / DIMM 184 pines	200 a 400 MT/s 1.6 a 3.2 GB/s	Módulos DDR-266 / DDR-400
2003	DDR2 SDRAM	Prefetch de 4n, bus E/S al doble del núcleo, ODT	1.8 V / DIMM 240 pines	400 a 800 MT/s 3.2 a 6.4 GB/s	DDR2-667 / DDR2-800
2007	DDR3 SDRAM	Prefetch de 8n, bajo voltaje, señalización mejorada	1.5 V / 1.35 V DIMM 240 pines	800 a 2133 MT/s 6.4 a 17.0 GB/s	Módulos DDR3-1600 / DDR3-1866
2014	DDR4 SDRAM	Prefetch 8n con Bank Groups, señal POD, conector curvo	1.2 V / DIMM 288 pines	1600 a 3200 MT/s 12.8 a 25.6 GB/s	DDR4-2400 / DDR4-3200
2020	DDR5 SDRAM	Dos subcanales de 32 bits, PMIC en módulo, On-Die ECC	1.1 V / DIMM 288 pines	4800 a 8400+ MT/s 38.4 a 67.2+ GB/s	DDR5-4800 / DDR5-6000+
2015+	HBM / HBM2e	Apilamiento 3D con vías TSV sobre interposer pasivo	1.2 V / Bus 1024–4096 bits	Hasta 3.2+ Gbps/pin > 1000 GB/s	Aceleradores GPU e IA

3. Evolución histórica de la memoria secundaria y almacenamiento masivo

3.1. Tecnologías electromecánicas y magnéticas tempranas

La incapacidad de la memoria principal para retener estados en ausencia de suministro eléctrico ininterrumpido impulsó el desarrollo de subsistemas de almacenamiento secundario persistente masivo (Deitel et al., 2004, §2.3, p. 66). Los primeros esquemas descansaron en soportes electromecánicos basados en papel perforado, desde la tarjeta perforada de Herman Hollerith (1890) estandarizada por IBM en 1928 (80 columnas de 12 filas rectangulares), sirviendo como medio primario de entrada, salida y resguardo secuencial durante la primera mitad del siglo XX.

La transición hacia medios reescribibles se materializó con la cinta magnética en el subsistema UNISERVO de la computadora UNIVAC I (1951) (Deitel et al., 2004, §12.1, p. 528). Basada en tiras de mylar con partículas de óxido de hierro (Fe_2O_3), ofrecía gran capacidad a bajo costo, pero su naturaleza lineal obligaba al rebobinado físico, siendo un acceso estrictamente secuencial que persiste hoy en cartuchos de resguardo terciario LTO (*Linear Tape-Open*) (Silberschatz et al., 2006, §1.8, p. 21). Para lograr acceso directo sin desplazamiento lineal, Gustav Tauschek concibió en 1932 el tambor magnético (adoptado en la IBM 650 en los años cincuenta), consistente en un cilindro ferromagnético giratorio con cabezales fijos por pista, aunque con capacidad limitada por el área superficial cilíndrica.

La génesis del almacenamiento secundario moderno con acceso aleatorio directo ocurrió el 4 de septiembre de 1956 con el sistema de almacenamiento en disco rígido IBM 305 RAMAC (*Random Access Method of Accounting and Control*) (Deitel et al., 2004, §12.1, p. 528). Diseñado por el equipo de Reynold B. Johnson, sustituyó el cilindro por un apilamiento vertical de 50 discos de aluminio de 24 pulgadas girando a 1200 RPM, tal como se documenta en la Img. 6.

Un único brazo gobernado por servoneumáticos se desplazaba verticalmente para seleccionar el plato y radialmente para posicionar el cabezal sobre un cojín de aire a presión. Con una capacidad de 5 millones de caracteres (≈ 5 MB) y una latencia media de 600 ms, el RAMAC consagró la superioridad del acceso directo y sentó las bases del disco duro moderno (Deitel et al., 2004, §12.1, p. 528).

3.2. Medios ópticos y consolidación del disco duro magnético (HDD)

Durante las décadas posteriores al RAMAC, el disco duro magnético (*hard disk drive*, HDD) experimentó una dramática miniaturización e incremento de densidad superficial. En soportes flexibles surgieron los disquetes (*floppy disks*): desarrollados por IBM en 1971 en 8 pulgadas (80 KB), derivados en 1976 a 5.25 pulgadas (160 KB a 1.2 MB) y consolidados por Sony en 1981 en 3.5 pulgadas (1.44 MB HD en 1987) como estándar universal de intercambio.

La arquitectura del disco duro magnético contemporáneo se consolidó en una estructura mecánica sellada con múltiples platos rígidos a velocidad angular constante (CAV: 5400 a 15000 RPM) (Silberschatz et al., 2006, §12.1, pp. 409–411; Deitel et al., 2004, §12.3, p. 529), ilustrada en la Img. 7.

Los cabezales con sensores magnetorresistivos túnel (TMR) vuelan a menos de 10 nm de la superficie impulsados por un motor de bobina móvil (*voice coil motor*, VCM). La la-

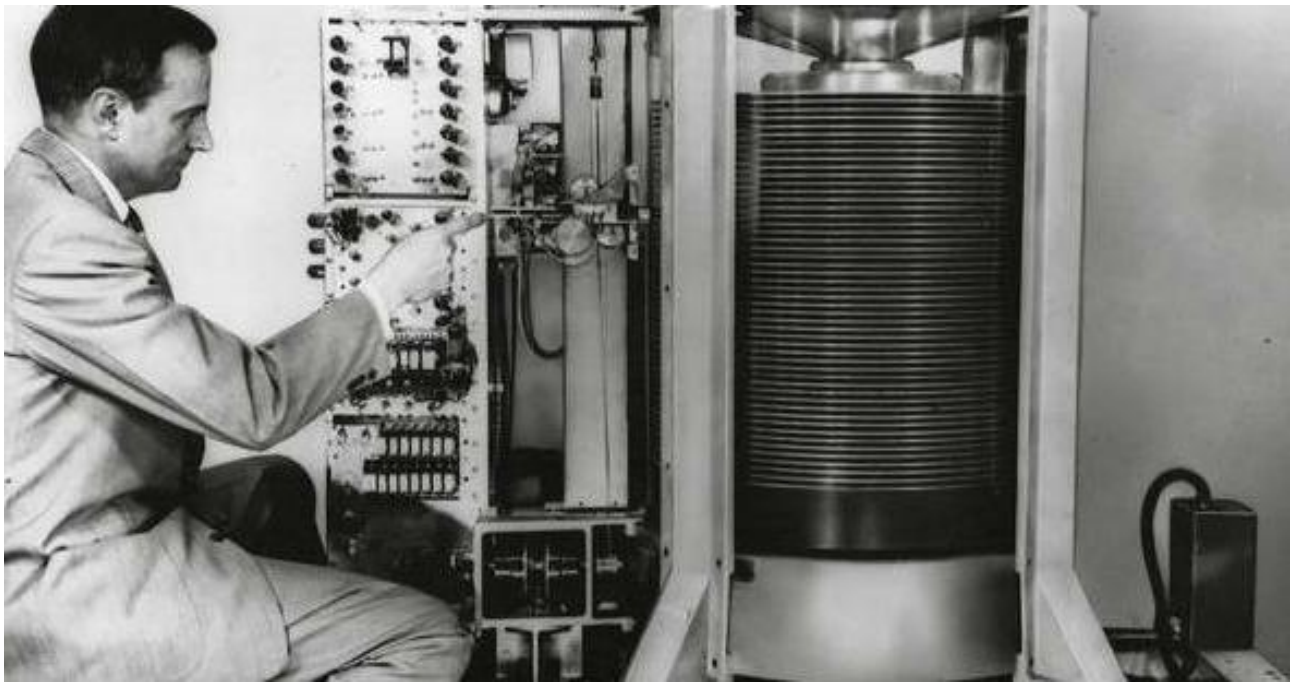


Imagen 6: Sistema de almacenamiento en disco rígido IBM 305 RAMAC con 50 platos de 24 pulgadas (1956) (Deitel et al., 2004, §12.1, p. 528).

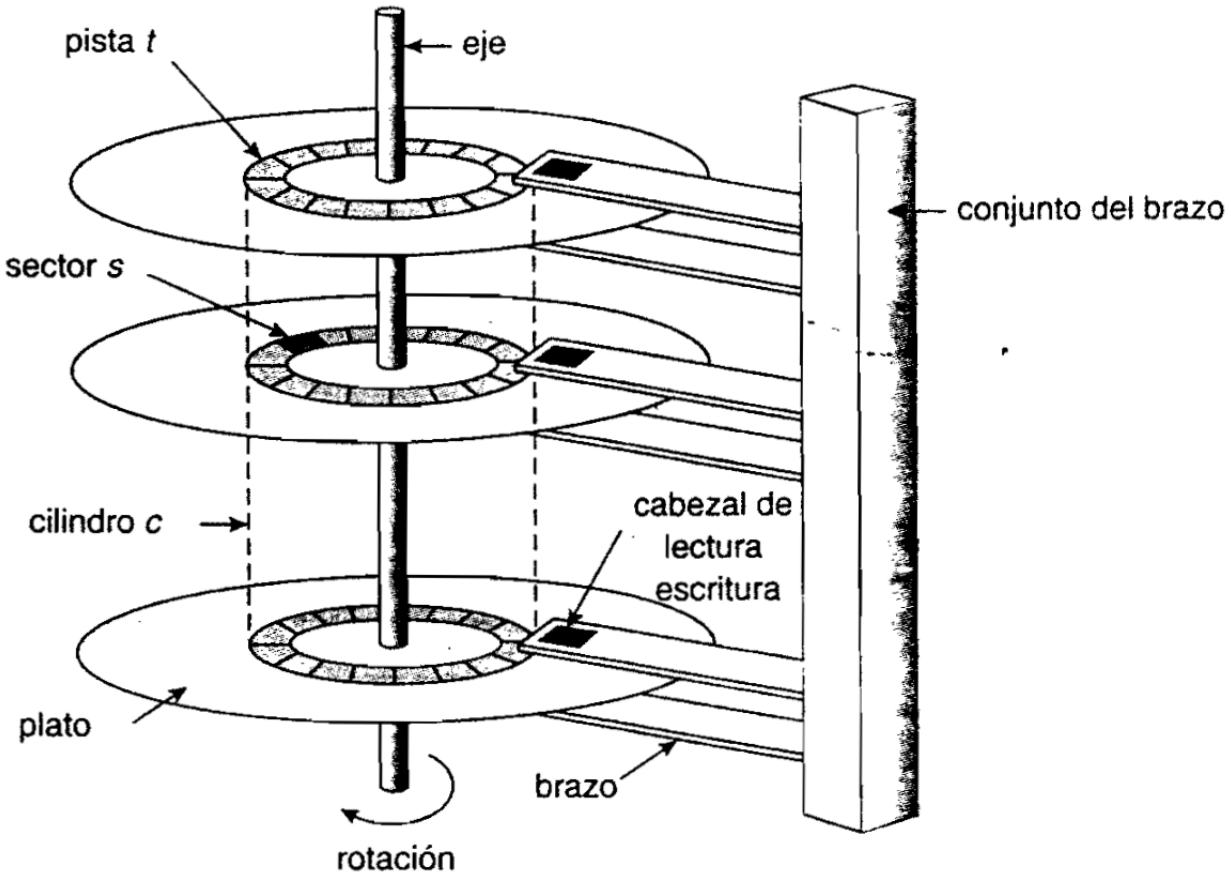


Imagen 7: Anatomía interna de un disco rígido magnético contemporáneo (platos, cabezales, cilindros y actuador VCM) (Silberschatz et al., 2006, §12.1, p. 409).

tencia total de acceso responde a tres parámetros temporales (Stallings, 2018, §11.5, pp. 518–519):

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN} \tag{2}$$

donde T_s es el tiempo medio de búsqueda (*seek time*, 3 a 9 ms); $\frac{1}{2r}$ es el retardo rotacional medio (*rotational latency*, ej., 4.16 ms a 7200 RPM); y $\frac{b}{rN}$ es el tiempo de transferencia de b bytes en una pista de N bytes. Lógicamente, el direccionamiento evolucionó de coordenadas físicas CHS a bloques lógicos LBA (*Logical Block Addressing*, sectores de 512B o 4Kn), interconectados mediante SATA III (6 Gbps / AHCI) y SAS (*Serial Attached SCSI*, 12 Gbps), con grabación perpendicular PMR/SMR y sellado con gas helio (Silberschatz et al., 2006, §12.2, p. 410). En paralelo, surgieron medios ópticos de velocidad lineal constante (CLV) leídos por láser: Compact Disc (CD-ROM, 1982, láser infrarrojo de 780 nm, 700 MB), Digital Versatile Disc (DVD, 1995, láser rojo de 650 nm, 4.7 a 8.5 GB) y Blu-ray Disc (BD, 2006, láser azul-violeta de 405 nm, 25 a 100 GB).

3.3. Tecnologías de estado sólido y almacenamiento no volátil contemporáneo

La barrera mecánica de los HDD (≈ 3 a 10 ms) motivó la transición hacia el estado sólido basado en memoria Flash, concebida por Fujio Masuoka en Toshiba (1984 Flash NOR, 1987 Flash NAND) (Deitel et al., 2004, §12.1, p. 528). La celda Flash NAND almacena carga por tunelamiento cuántico Fowler-Nordheim en compuertas flotantes (*floating-gate*) o trampas de nitruro de silicio (*charge-trap*), alterando la tensión umbral V_{th} sin requerir energía externa (Silberschatz et al., 2006, §1.2, p. 9). La densidad escaló por cuantización multinivel: SLC (1 bit/celda), MLC (2 bits), TLC (3 bits) y QLC (4 bits/celda), y por apilamiento tridimensional (3D NAND / V-NAND con más de 200 capas).

Un SSD (*solid-state drive*) moderno integra controlador microprogramado ASIC, DRAM caché y canales Flash paralelos, como se detalla en la Img. 8.

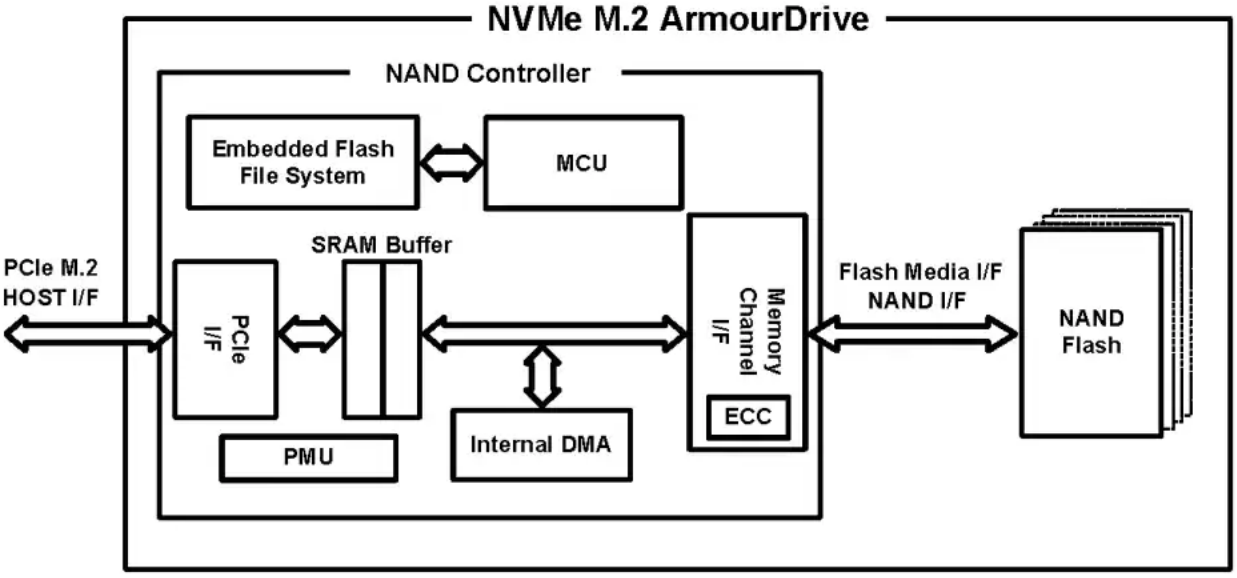


Imagen 8: Diagrama de bloques de un SSD NVMe M.2 con controlador FTL, DRAM y canales Flash 3D NAND (Silberschatz et al., 2006, §1.8, p. 22).

La capa de traducción de Flash (*Flash Translation Layer*, FTL) gestiona la asimetría de la Flash NAND: lectura/escritura por páginas (4–16 KB) y borrado por bloques completos (2–8 MB) (Silberschatz et al., 2006, §1.2, p. 9). Al prohibirse la sobreescritura directa (*no in-place overwrite*), la FTL ejecuta: escritura fuera de lugar con mapeo LBA-PBA; recolección de basura (*garbage collection*) de bloques obsoletos; nivelación de desgaste (*wear leveling*) homogéneo; y procesamiento del comando TRIM / Deallocate del sistema operativo para descartar páginas eliminadas y abatir el factor de amplificación de escritura (WAF). La interfaz NVMe (*Non-Volatile Memory Express*) sobre bus PCIe elimina cuellos de botella con hasta 64 000 colas paralelas de 64 000 comandos cada una (Silberschatz et al., 2006, §1.8, p. 22). La Tabla 3 resume la cronología evolutiva del almacenamiento secundario.

En interconexión, los primeros SSD adoptaron SATA III con protocolo AHCI, que concebido para discos magnéticos rotatorios limitaba transferencias a una cola de 32 comandos y 600 MB/s. La superación de este cuello de botella provino del protocolo NVMe (*Non-Volatile Memory Express*), diseñado para operar sobre el bus PCI Express (PCIe 3.0, 4.0 y 5.0). NVMe reemplaza la cola única de AHCI con una arquitectura escalable de hasta 64,000 colas con 64,000 comandos cada una, reduciendo sustancialmente ciclos de CPU por E/S y elevando transferencias secuenciales a 7,000–14,000 MB/s en factores compactos M.2 y U.2/EDSFF, con latencias de 10 a 20 microsegundos.

Finalmente, en el extremo más avanzado de la jerarquía no volátil emergieron memorias de cambio de fase como 3D XPoint (comercializada como Intel Optane), basada en una matriz de cruce de conductores (*cross-point*) sin transistores que conmutaba la resistencia de un material calcogenuro. Al brindar direccionamiento por byte y lecturas/escrituras simétricas sin requerir ciclos de borrado de bloque, 3D XPoint operaba como memoria de clase almacenamiento (*storage-class memory*, SCM), cerrando la brecha de rendimiento entre la DRAM y la memoria Flash NAND.

La Tabla 3 detalla la cronología evolutiva integral de las tecnologías de memoria secundaria y almacenamiento masivo.

Tabla 3: Cronología evolutiva integral de las tecnologías de memoria secundaria y almacenamiento.

Año	Tecnología y denominación	Soporte físico y mecanismo	Capacidad / Factor de forma	Latencia / Velocidad típica	Hito o referencia de imagen
1890	Tarjetas perforadas	Cartulina con perforaciones físicas en matriz	80 columnas ≈ 80 bytes/tarjeta	Lectura mecánica 100–1000 tarj/min	Herman Hollerith IBM censo 1890
1951	Cinta magnética	Película de mylar con recubrimiento de óxido de hierro	Carrete $1/2''$ 1.4 MB por cinta	Acceso secuencial 100 bytes/pulgada	UNIVAC I UNISERVO
1953	Tambor magnético	Cilindro metálico giratorio con cabezales fijos por pista	Cilindros dedicados 10 a 60 KB	Retardo rotacional ≈ 10 a 20 ms	IBM 650 mainframe
1956	IBM 305 RAMAC	50 platos de aluminio de 24 pulgadas y cabezal neumático	5 MB (7 bits) Mueble masivo	Latencia ≈ 600 ms 8.8 KB/s	Img. 6 Primer HDD comercial
1971	Disquete (Floppy)	Disco flexible de mylar protegido en funda de plástico	8”, 5.25”, 3.5” 80 KB a 1.44 MB	Rotación a 300 RPM Latencia > 100 ms	IBM 3740 y Sony Almacenamiento portátil
1980	HDD 5.25” / 3.5”	Platos rígidos en rotación CAV y actuador de bobina VCM	ST-506, IDE/PATA 5 MB a 500 MB	Seek time de 15–40 ms Transferencia < 5 MB/s	Seagate ST-506 Estandarización de PC
1982	CD-ROM	Lectura óptica de pozos/planos con láser infrarrojo (780 nm)	Disco de 120 mm 650 a 700 MB	Modo CLV 1x 150 KB/s ($1200\,\mu$s)	Estándar Red/Yellow Book Philips / Sony
1995	DVD-ROM	Pistas ópticas estrechas leídas por haz láser rojo (650 nm)	Disco de 120 mm 4.7 GB a 8.5 GB	Modo CLV 1x 1.38 MB/s ($150\,\mu$s)	Consorcio DVD Video y software masivo
2000+	HDD SATA / SAS	Platos con grabación PMR/SMR y cavidad de helio	3.5” / 2.5” 1 TB a 24+ TB	Seek time de 4–8 ms 150 a 280 MB/s	Img. 7 Almacenamiento masivo
2006	Blu-ray Disc	Láser azul-violeta (405 nm) de alta apertura numérica	Disco de 120 mm 25 GB a 100 GB	Modo CLV 1x 4.5 MB/s (hasta 54 MB/s)	Distribución HD y BDXL
2008	SSD SATA (NAND)	Matrices Flash NAND con FTL sobre bus SATA III AHCI	Factor de forma 2.5” 64 GB a 4 TB	Latencia $\approx 100\,\mu$s Máximo 550 MB/s	Adopción de estado sólido
2015	SSD NVMe PCIe	Flash 3D NAND vertical sobre bus PCIe con colas paralelas	Factor M.2 / U.2 500 GB a 8+ TB	Latencia 10–20 μs 3500 a 14000 MB/s	Img. 8 PCIe 3.0 / 4.0 / 5.0
2015	3D XPoint	Memoria de cambio de fase resistiva sin borrado de bloque	M.2 / Add-In Card 16 GB a 1.5 TB	Latencia $< 10\,\mu$s Ultra alta durabilidad	Intel Optane / Micron Storage-Class Memory

4. Memoria de intercambio (Swapping) y gestión de memoria virtual

4.1. Definición y distinción técnica: intercambio de procesos frente a paginación bajo demanda

En la ingeniería de sistemas operativos, la memoria de intercambio (*swapping*) es el mecanismo mediante el cual el núcleo transfiere temporalmente regiones del espacio de direccionamiento desde la memoria física primaria (DRAM) hacia un almacén de respaldo (*backing store*) secundario, liberando marcos físicos y restituyendo la información al reanudarse la tarea (Silberschatz et al., 2006, §8.2, p. 249). Arquitectónicamente, debe distinguirse entre el **intercambio clásico** de procesos (*roll out / roll in*) en particiones continuas, donde el kernel trasladaba en bloque la imagen íntegra del proceso (código, datos, montón y pila) con severas penalizaciones temporales de E/S que elevaban el costo del cambio de contexto a niveles prohibitivos (Stallings, 2018, §7.2, p. 343), y la **paginación bajo demanda** (*demand paging*) asistida por la MMU contemporánea, donde el intercambio opera a nivel de páginas virtuales de 4 KB (Carretero Pérez et al., 2001, §4.6, p. 165), cargadas bajo fallo de página (*page fault*) y desalojadas selectivamente a disco (*page-out*) sin transferir estructuras completas (Silberschatz et al., 2006, §9.2, p. 283).

El subsistema clasifica las páginas según su origen de respaldo: las *páginas respaldadas por archivos* (*file-backed*), como código ejecutable o bibliotecas compartidas, que al ser desalojadas no requieren escribirse al swap si permanecen limpias (*clean*), pues se recargan del binario original; y la *memoria anónima* (*anonymous memory*), integrada por la pila (*stack*), montón (*heap*) y variables modificadas, que al carecer de respaldo persistente deben escribirse obligatoriamente en el área de intercambio para evitar la pérdida del estado de cómputo (Silberschatz et al., 2006, §12.4, p. 415).

4.2. Funciones y operatividad de la memoria de intercambio en el sistema operativo

La memoria de intercambio cumple tres funciones operativas cardinales en el núcleo (Stallings, 2018, §8.1, p. 371): 1) **Ampliación del espacio de direccionamiento**, proyectando una memoria virtual global superior a la DRAM física para admitir cargas analíticas pesadas (Deitel et al., 2004, §10.3, p. 432); 2) **Regulación de la multiprogramación**, permitiendo suspender procesos dormidos o bloqueados en E/S prolongada y enviar sus páginas inactivas a disco para hacer espacio a tareas listas (Stallings, 2018, §3.2, p. 136); y 3) **Optimización de la caché de páginas (*page cache*)**, donde el kernel expulsa páginas anónimas infrecuentes para ceder marcos libres a la caché de archivos, acelerando la E/S global del sistema (Carretero Pérez et al., 2001, §4.9, p. 181).

Estructuralmente, el área de intercambio opera como una *partición dedicada* (*swap partition*) sin sobrecarga de inodos mediante mapa de ranuras (*swap map*), o como un *archivo de intercambio* (*swap file*, ej., `pagefile.sys` o montajes vía `swapon`) de redimensionamiento dinámico (Silberschatz et al., 2006, §12.4, p. 416). Cada ranura (*swap slot*) equivale a una página física y mantiene contadores de referencia para páginas compartidas (*copy-on-write*) (Silberschatz et al., 2006, §9.3, p. 289). El desalojo es modulado por la sobreasignación (*memory overcommit*) y la afinidad (*swappiness*), que balancea de 0 a 100 el rescate de páginas de caché frente al swap anónimo.

4.3. Rendimiento, degradación por hiperpaginación (*Thrashing*) y mecanismos de mitigación

La hiperpaginación (thrashing) ocurre cuando el conjunto agregado de procesos activos no dispone de marcos físicos suficientes para mantener sus páginas de uso frecuente, provocando una cascada continua de fallos de página (Silberschatz et al., 2006, §9.6.1, p. 305). En este estado, los procesos seleccionados por la CPU detienen su ejecución esperando lecturas de swap; el despachador interpreta la subutilización de CPU como baja carga e introduce más procesos, colapsando el bus de E/S y reduciendo el rendimiento útil a cero (Stallings, 2018, §8.2, p. 388).

Para erradicar la hiperpaginación, Peter J. Denning formuló en 1968 el modelo del conjunto de trabajo (*working set model*) basado en el principio de localidad (Silberschatz et al., 2006, §9.6.2, p. 307). Definiendo una ventana temporal de referencias recientes Δ, el conjunto de trabajo $W_i(t, \Delta)$ contiene las páginas virtuales referenciadas por el proceso i en $[t - \Delta, t]$. La demanda total de marcos corresponde a:

$$D = \sum_{i=1}^{n} |W_i(t, \Delta)| \tag{3}$$

donde $|W_i(t, \Delta)|$ es la cardinalidad de páginas activas del proceso i. La estabilidad exige que la demanda no exceda los marcos físicos disponibles ($D \leqslant M$); si $D > M$, el planificador debe suspender y expulsar procesos completos a disco hasta restaurar el balance (Stallings, 2018, §8.2, p. 401). Complementariamente, la política de frecuencia de fallos de página (*page-fault frequency*, PFF) asigna o confisca marcos monitoreando umbrales dinámicos τ_{inf} y τ_{sup} (Silberschatz et al., 2006, §9.6.3, p. 308). Ante agotamiento crítico, mecanismos como el Asesino de Fuera de Memoria (*Out-Of-Memory Killer*, OOM Killer) calculan una función de nocividad (*badness score*) para terminar forzosamente mediante `SIGKILL` el proceso más gravoso, preservando la estabilidad del núcleo (Deitel et al., 2004, §11.6, p. 494).

5. Arquitectura y ciclo de vida de los procesos

5.1. Definición formal de proceso y organización en memoria

En la teoría de sistemas operativos, un proceso se define formalmente como un programa en ejecución, constituyendo la entidad activa fundamental de cómputo y la unidad elemental de asignación de recursos protegidos del procesador (Silberschatz et al., 2006, §3.1.1, p. 73). Esta definición establece una demarcación rigurosa frente al concepto de programa: mientras que un programa es una entidad estática y pasiva almacenada en soporte no volátil como una secuencia binaria estructurada bajo formatos como ELF (*Executable and Linkable Format*) o PE (*Portable Executable*), un proceso es una entidad dinámica caracterizada por un hilo de control, un espacio virtual exclusivo, estados de ejecución cambiantes y posesión activa de recursos del sistema (Stallings, 2018, §3.1, p. 131).

El espacio de direccionamiento virtual asignado a un proceso por el núcleo se descompone en cinco regiones o segmentos contiguos en memoria lógica (Carretero Pérez et al., 2001, §3.3.2, p. 103): 1) **Segmento de texto (*text segment*)**, con instrucciones de máquina en direcciones bajas con permisos de sólo lectura y ejecución, dotado de reentrancia para compartir marcos físicos; 2) **Segmento de datos inicializados (*data segment*)**, reservado para variables globales y estáticas con valor inicial explícito; 3) **Segmento de datos no inicializados (*Block Started by Symbol*, BSS)**, para variables globales sin inicializar, asignadas a cero por el cargador del kernel sin ocupar espacio en disco (Carretero Pérez et al., 2001, §3.3.2, p. 104); 4) **Montón (heap)**, zona de asignación dinámica solicitada por llamadas como `brk` o `malloc`, cuyo límite crece hacia direcciones altas (Silberschatz et al., 2006, §3.1.1, p. 74); y 5) **Pila de llamadas (call stack)**, estructura LIFO en direcciones superiores que desciende hacia el montón, almacenando marcos de activación con parámetros, retornos y variables locales (Stallings, 2018, §3.3, p. 151). Una franja intermedia no mapeada previene colisiones accidentales entre pila y montón.

5.2. Estructura y características del proceso: Bloque de Control de Proceso (PCB)

El comportamiento operativo del proceso exhibe cuatro características cardinales (Carretero Pérez et al., 2001, §3.1, p. 97): 1) **Concurrencia**, permitiendo intercalar su ejecución con otros flujos en el procesador; 2) **Asincronía y no determinismo**, dado que su progreso depende de latencias de E/S y decisiones del planificador; 3) **Aislamiento y protección espacial**, garantizados por la MMU mediante tablas de páginas independientes que prohíben accesos ilícitos ajenos (Stallings, 2018, §3.1, p. 132); y 4) **Posesión de recursos**, fungiendo como titular de descriptores de archivos, memoria y cuotas de cómputo.

Para gobernar estos recursos, el sistema materializa cada proceso mediante el Bloque de Control de Proceso (*Process Control Block*, PCB; `task_struct` en Linux) (Silberschatz et al., 2006, §3.1.3, p. 75), estructurado en cinco áreas clave (Stallings, 2018, §3.3, p. 152): 1) **Identificación**, con identificador de proceso (PID), del padre (PPID) y credenciales UID/GID; 2) **Contexto de hardware**, con contador de programa (PC), puntero de pila (SP), registros generales, banderas y registro de tabla de páginas (CR3 en x86) para conmutación (Silberschatz et al., 2006, §3.1.3, p. 76); 3) **Planificación**, con estado y colas del despachador; 4) **Memoria**, con descriptores virtuales y tablas de páginas (`mm_struct`); y 5) **Contabilidad y E/S**, con descriptores de archivos, dispositivos y consumo de CPU.

5.3. Modelo de estados y transiciones en la ejecución del proceso

El ciclo vital de un proceso transita por estados discretos formalizados por el modelo clásico de cinco estados (Stallings, 2018, §3.2, p. 133): **Nuevo (*New*)**, durante la inicialización del PCB y asignación preliminar de descriptores (Silberschatz et al., 2006, §3.1.2, p. 74); **Listo (*Ready*)**, con todas las páginas residentes en memoria física esperando turno de CPU; **En ejecución (*Running*)**, con instrucciones ejecutándose activamente en los registros del procesador; **Bloqueado (*Blocked*)**, aguardando la finalización de una operación de E/S o un evento de sincronización (Carretero Pérez et al., 2001, §3.5, p. 115); y **Terminado (*Exit*)**, concluido mediante la llamada `exit`, reteniendo su PCB en estado zombi (*zombie*) hasta que el padre recolecta su código de terminación mediante `wait` (Silberschatz et al., 2006, §3.3.2, p. 85).

La dinámica de ejecución se articula mediante transiciones gobernadas por el planificador del kernel (Stallings, 2018, §3.2, p. 134): 1) *Admisión* (Nuevo → Listo), cuando el sistema valida la disponibilidad de memoria; 2) *Despacho* (Listo → Ejecución), al asignar el procesador según la política de turnos; 3) *Expropiación o fin de cuanto* (Ejecución → Listo), gatillada por la interrupción periódica del reloj del sistema (*timer interrupt*) en esquemas apropiativos; 4) *Bloqueo* (Ejecución → Bloqueado), cuando el proceso emite una llamada al sistema bloqueante; y 5) *Despertar* (Bloqueado → Listo), originado por una interrupción de hardware que notifica la disponibilidad del periférico.

Ante saturación severa de memoria física y riesgo de hiperpaginación, Stallings formula el *modelo de siete estados*, integrando el almacenamiento secundario con dos estados de suspensión (Stallings, 2018, §3.2, p. 138): **Listo-suspendido (*Ready/Suspend*)**, el cual se encuentra en condiciones de ejecutar pero reside temporalmente en el área de swap; y **Bloqueado-suspendido (*Blocked/Suspend*)**, que aguarda un evento de E/S mientras su imagen permanece desalojada a disco. Este modelo introduce la optimización de *resolución de evento en swap*: al concluir la operación de E/S externa, el controlador del dispositivo promueve el proceso directamente desde Bloqueado-suspendido hacia Listo-suspendido sin forzar la recarga inmediata de sus marcos a la DRAM, permitiendo al planificador de mediano plazo evaluar estratégicamente el momento óptimo de reinserción según la carga global del sistema (Stallings, 2018, §3.2, p. 140).

6. Hilos de ejecución y análisis comparativo frente a procesos

6.1. Definición formal de hilo (*Thread*) y estructura de control (TCB)

En la ingeniería de sistemas operativos, un hilo de ejecución (thread, hebra o proceso ligero) se define formalmente como la unidad elemental de despacho y utilización de la CPU, constituyendo un flujo secuencial independiente dentro del espacio de un proceso (Silberschatz et al., 2006, §4.1, p. 111). Esta conceptualización formaliza una dualidad arquitectónica: el proceso representa la unidad de **propiedad de recursos** y protección espacial, mientras que el hilo constituye la unidad de **planificación y ejecución temporal** (Stallings, 2018, §4.1, p. 177).

Los hilos de un proceso comparten homogéneamente su espacio virtual (texto, datos, BSS y montón), descriptores de archivos, sockets y manejadores de señales (Silberschatz et al., 2006, §4.1, p. 112). Cada hilo preserva autonomía mediante elementos privados exclusivos: identificador de hilo (TID), contexto volátil de registros (PC, SP y registros generales), pila de llamadas propia (*per-thread stack*) y almacenamiento local (TLS). El núcleo materializa esta entidad mediante el Bloque de Control de Hilo (*Thread Control Block*, TCB) (Carretero Pérez et al., 2001, §3.6, p. 121); en Linux, la gestión converge en `task_struct` mediante la llamada `clone()` con banderas de compartición (`CLONE_VM`, `CLONE_FILES`, `CLONE_SIGHAND`) (Stallings, 2018, §4.6, p. 208).

6.2. Modelos de gestión de hilos y correspondencia de planificación

La administración de hilos se bifurca según el nivel arquitectónico que gestiona su ciclo de vida (Stallings, 2018, §4.2, p. 183): los **hilos a nivel de usuario** (*User-Level Threads*, ULT), coordinados por bibliotecas en espacio de usuario sin intervención del kernel, ofrecen conmutación rápida sin cambio de modo de privilegio (*mode switch*), pero carecen de paralelismo multinúcleo y bloquean al proceso completo ante una llamada al sistema síncrona; y los **hilos a nivel de núcleo** (*Kernel-Level Threads*, KLT), despachados por el planificador del kernel, que habilitan ejecución paralela simétrica y tolerancia a bloqueos de E/S a cambio de una mayor sobrecarga de transición al modo supervisor (Stallings, 2018, §4.2, p. 187). La correspondencia entre ambas capas se articula mediante tres modelos (Silberschatz et al., 2006, §4.2, pp. 114–117): **Muchos a uno ($N : 1$)**, con múltiples ULT sobre un único KLT; **Uno a uno ($1 : 1$)**, estándar de facto en Linux (NPTL) y Windows que asocia cada ULT a un KLT dedicado; y **Muchos a muchos ($M : N$)**, que multiplexa dinámicamente M hilos de usuario sobre N hilos de núcleo ($M \geqslant N$).

6.3. Estudio comparativo y diferencial entre procesos e hilos

El contraste entre procesos pesados e hilos abarca dimensiones críticas de desempeño, arquitectura y diseño de software (Stallings, 2018, §4.1, p. 179). En el ámbito de la instanciación, crear un proceso mediante la llamada al sistema `fork()` impone una sobrecarga notable: el núcleo debe duplicar o mapear estructuras de tablas de páginas, reservar descriptores de archivos, asignar un nuevo espacio de direcciones virtuales y configurar cuotas de planificación, requiriendo típicamente entre 10 y 30 veces más ciclos de procesador que instanciar un hilo mediante `pthread_create()`, el cual únicamente exige reservar la estructura TCB y asignar una pila privada en el montón del proceso (Stallings, 2018, §4.1, p. 179).

En la conmutación de contexto, alternar la CPU entre procesos distintos exige invalidar o actualizar los registros del búfer de traducción anticipada (TLB) y modificar el registro de control de la tabla de páginas (como el registro CR3 en la arquitectura x86), provocando una degradación temporal del rendimiento por fallos de caché fríos. En contraparte, conmutar entre hilos del mismo proceso preserva íntegramente las traducciones de la memoria virtual activa en el TLB y las líneas de caché de datos e instrucciones, reduciendo la penalización al simple guardado y restauración de los registros de trabajo del procesador y del puntero de pila (Carretero Pérez et al., 2001, §3.5.1, p. 119).

En el paradigma de comunicación interna, los procesos demandan mecanismos formales de comunicación entre procesos (IPC) como tuberías (*pipes*), colas de mensajes, memoria compartida (`shmget`) o sockets de red locales, requiriendo validaciones de seguridad por el kernel y copias intermedias de búferes (Silberschatz et al., 2006, §3.4, pp. 86–90). Por el contrario, los hilos se comunican de forma instantánea leyendo y modificando variables globales y estructuras en el montón común. No obstante, esta accesibilidad introduce el riesgo de condiciones de carrera (*race conditions*), forzando al programador a incorporar primitivas explícitas de sincronización (cerrojos de exclusión mutua *mutex*, semáforos contadores de Dijkstra y variables de condición) sustentadas en instrucciones atómicas del procesador como *Compare-And-Swap* (CAS) (Silberschatz et al., 2006, §4.1, p. 112).

Finalmente, en cuanto a tolerancia a fallos y seguridad, el proceso provee una barrera de aislamiento infranqueable garantizada por la MMU: una excepción fatal o fallo de segmentación (`SIGSEGV`) queda confinado estrictamente a su contexto. En cambio, al carecer de aislamiento de memoria entre sí, un puntero erróneo o violación de acceso en un único hilo desencadena el colapso inmediato e integral de todo el proceso contenedor. La Tabla 4 sintetiza de forma estructurada las divergencias operativas y arquitectónicas entre ambos esquemas.

Tabla 4: Estudio comparativo y diferencial entre procesos pesados e hilos de ejecución.

Criterio diferencial	Proceso (Proceso pesado)	Hilo de ejecución (Proceso ligero)
Rol arquitectónico	Unidad formal de posesión de recursos, contabilidad y protección espacial (Stallings, 2018, §4.1, p. 177).	Unidad elemental de despacho de procesador, planificación y flujo de ejecución (Silberschatz et al., 2006, §4.1, p. 111).
Espacio de memoria	Espacio de direcciones virtuales privado e independiente, aislado por tablas de la MMU.	Comparte el espacio virtual del proceso contenedor (texto, datos, BSS, montón) con pila propia.
Estructura de control	Bloque de Control de Proceso (PCB / `task_struct`), con descriptores y mapa de memoria.	Bloque de Control de Hilo (TCB), limitado a identificador (TID), registros volátiles, PC y SP.
Sobrecarga de creación	Elevada: exige replicar o mapear estructuras de páginas, descriptores de archivos y cuotas de cómputo.	Mínima: requiere únicamente reservar el TCB y una pila privada (10 a 30 veces más rápida) (Stallings, 2018, §4.1, p. 179).
Cambio de contexto	Penalización temporal alta: requiere invalidar el búfer TLB y conmutar el registro CR3 de la tabla de páginas.	Conmutación liviana: preserva las entradas del TLB y la tabla de páginas activa; sólo conmuta registros y SP (Carretero Pérez et al., 2001, §3.5.1, p. 119).
Comunicación interna	Exige mecanismos de IPC (tuberías, colas de mensajes, memoria compartida `shmget`, sockets de red).	Comunicación directa por variables globales en el montón; exige cerrojos mutex y semáforos (Silberschatz et al., 2006, §4.1, p. 112).
Aislamiento y fallos	Protección espacial total: una excepción fatal o fallo de segmentación queda confinado al proceso.	Sin aislamiento de memoria: un puntero erróneo o violación de acceso en un hilo aborta el proceso íntegro.

7. Conclusiones personales

El estudio sistemático de la memoria y la gestión de procesos demuestra que el diseño de los sistemas operativos descansa en una continua mediación entre abstracciones de software y restricciones del hardware. La jerarquía de memoria surge como una respuesta imperativa a la brecha insalvable entre la velocidad de la CPU y las latencias de almacenamiento. Desde tecnologías pioneras como los tubos Williams y núcleos de ferrita magnética hasta la consolidación de la DRAM y los discos NVMe de estado sólido, cada avance ha mitigado el cuello de botella de von Neumann. En este esquema, la unidad de manejo de memoria (MMU) y la virtualización del direccionamiento constituyen los pilares que otorgan a cada tarea la ilusión de disponer de un espacio contiguo, uniforme y protegido contra interferencias externas.

Por su parte, la memoria de intercambio (swapping) y la paginación bajo demanda evidencian que la memoria física no limita de forma estricta el grado de multiprogramación. Al trasladar selectivamente páginas inactivas y memoria anónima hacia el disco, el núcleo maximiza el aprovechamiento de la memoria principal y optimiza la memoria caché de archivos. Sin embargo, este balance es delicado: la hiperpaginación (thrashing) ejemplifica cómo una sobreasignación de memoria o un deficiente seguimiento del conjunto de trabajo anulan el ancho de banda útil del procesador. El empleo de mecanismos adaptativos como la política de frecuencia de fallos de página y salvaguardas de última instancia como el OOM Killer subraya el rigor matemático necesario para prevenir la degradación catastrófica del sistema.

Finalmente, la dualidad entre procesos e hilos consolida los paradigmas modernos de concurrencia. Mientras el proceso proporciona una frontera infranqueable de aislamiento espacial, posesión de recursos y contabilidad protegida por hardware, el hilo aporta la agilidad requerida para explotar procesadores multinúcleo mediante conmutaciones de contexto livianas y memoria compartida. Como futuros ingenieros en computación, dominar estos conceptos es crucial, pues la elección entre la robustez del aislamiento por procesos y la eficiencia cooperativa de los hilos determina directamente la estabilidad, el desempeño y la escalabilidad de cualquier plataforma computacional.

Referencias

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson Prentice Hall.

McHoes, A. M., & Flynn, I. M. (2011). *Sistemas operativos* (6.ª ed.). Cengage Learning Editores.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.